Generalizable Facial Expression Recognition

A Mini Project as a Course requirement for

Master of Science

(Data Science and Computing)

Sai Narendra

24040208002

Sri Sathya Sai Institute of Higher Learning

(Deemed to be University)

Department of Mathematics and Computer Science

Nandigiri Campus

April 2025

Dept. of Mathematics & Computer Science
Nandigiri Campus

CERTIFICATE

This is to certify that this Mini Project titled **Generalizable Facial Expression Recognition** submitted by **G Sai Narendra**, Regd No. **24040208002**, Department of Mathematics and Computer Science, Nandigiri Campus is a bonafide record of the original work done under our supervision as a Course requirement for the Degree of Master of Science Data Science and Computing.

Countersigned by

_____
Prof.SVSN. Sarma
Supervisor

_____
Dr. Darshan Gera
Joint Supervisor

_____
Head of Department
Lakshmi Naidu

Place: Nandigiri
Date:

DECLARATION

The Mini Project titled **Generalizable Facial Expression Recognition** was carried out by me under the supervision of Dr. Darshan Gera and Prof. Sri SVSN Sarma as a Course requirement for the Degree of Masters of Science in Data Science and Computing and has not formed the basis for the award of any degree, diploma or any other such title by this or any other University.

Place: Nandigiri
Date:

_____
Sai Narendra 24040208002
MSc (Data Science and Computing)

Acknowledgments

First and foremost, I would like to express my most heartfelt gratitude to Bhagawan Sri Sathya Sai Baba for guiding my life to a point where I could take up a mini-project like this. From my education to my mental, physical, and spiritual well-being, He has been the sole caretaker of everything. I dedicate this work at His Divine Lotus Feet.

I would like to thank my project supervisor Prof. SVSN Sarma Sir for his unwavering support and boundless enthusiasm. His willingness to help me and listen to my problems was essential to this mini-projects progress.

I thank my external supervisor Dr. Darshan Gera. He came off as an extremely passionate man from our very first meeting, and it was thanks to this passion and “can do will do” attitude that I found the motivation to strive for better results in this mini-project. I would not have been able to make much progress without his expertise in the field of Computer Vision and the domain of Facial Expression Recognition.

I would like to acknowledge the perennial support I have received from my family and friends. My fellow batchmates and our impromptu discussions were the birthplace of many of the ideas I ended up implementing.

Conclusively, I would like to express my deepest gratitude to the Sri Sathya Sai Institute of Higher Learning for giving me everything that made this mini-project possible.

Abstract

State-of-the-art (SOTA) facial expression recognition (FER) methods struggle with domain gaps between training and test sets, limiting their generalization ability. Recent domain adaptation techniques require labeled or unlabeled target domain samples for fine-tuning, which may be infeasible in real-world deployment.

This project proposes a novel zero-shot generalization approach that enhances FER performance on unseen test sets using only a single training set. Inspired by the human perception process—first detecting faces and then recognizing expressions.

we introduce a FER pipeline that extracts expression-related features from face images. Our method leverages generalizable face features from large-scale models like CLIP while addressing the challenge of adapting them for FER.

To maintain CLIP's generalization capability and ensure high precision, we design a novel approach that learns sigmoid masks over fixed CLIP features to extract expression-relevant information.

Additionally, we improve generalization by separating feature channels based on expression classes, generating logits directly and reducing overfitting by avoiding fully connected layers. A channel-diverse loss is introduced to enhance feature separation.

Extensive experiments on two FER datasets demonstrate that our method significantly outperforms SOTA FER models.

Table of contents

1. Introduction

1.1 Motivation

Facial Expression Recognition (FER) is crucial for human-computer interaction and has advanced significantly with deep learning. However, FER models face domain gaps, struggling to generalize across different datasets. For example, a state-of-the-art (SOTA) model trained on RAF-DB performs well on its test set but poorly on other datasets like AffectNet. Existing domain generalization methods rely on labeled or unlabeled target data for fine-tuning, which is often impractical in real-world scenarios where test data distribution is unknown. This paper introduces **Generalizable Facial Expression Recognition (GFER)**, which enhances zero-shot generalization across unseen datasets using only a single training set without access to target samples.

1.2 Defining the Problem

In this mini-project, I attempt to build a Facial Expression Recognition (FER) model using Deep Learning techniques, that takes an image of a face (of a person) and classifies it as one of 8 basic expressions: Happy, Sad, Angry, Fearful, Disgusted, Surprised, Neutral, and Other. The 'Other' class contains expressions that do not fall under any of the basic expression classes.
The problem of Facial Expression Recognition comes with two primary dimensions:

- Static FER: The input features of our model only include the spatial information of a single image to make predictions on the expression.

In this mini-project, I only worked on Static FER, with plans to extend my work to Dynamic FER in the future.

1.3 Problem Statement (Gen FER 2024)

A lot of Facial Expression Recognition models already exist, each with diverse approaches. We decided to exploit the capabilities of OpenAI's CLIP model to extract image features and use these features to perform FER. My problem statement for this mini-project is:

Generalizable Facial Expression Recognition

Existing state-of-the-art (SOTA) FER methods struggle with unseen test sets due to overfitting to domain-specific information. To address this, we propose a novel pipeline inspired by human facial expression recognition. The method first extracts generalizable face features using pre-trained models like CLIP and then learns sigmoid masks to focus on expression-related features while preserving generalization ability.

Key innovations include:

1. Sigmoid Masks on Fixed Face Features – Maintains generalization by selecting expression-related information.
2. Channel-Separation Module – Reduces overfitting by organizing masked features according to expression-related channels.
3. Channel-Diverse Loss – Encourages diversity in learned masks, improving generalization.

2. Literature Review

2.1 CLIP Model (OpenAI, 2021)

CLIP (Contrastive Language-Image Pre-training) was developed by OpenAI in 2021. It is a model built to have Zero-Shot classification capabilities on previously unseen classes and datasets.

Trained on a dataset of 400 million image-text pairs collected from the internet, it essentially consists of two parts: an Image encoder, and a Vision encoder. The model is trained to project images and text into a vector space and maximize the cosine similarity of images and text pairs that are related and minimize the cosine similarity of images and text that are not related.

CLIP uses a ViT-like transformer to get visual features and a causal language model to get text features.

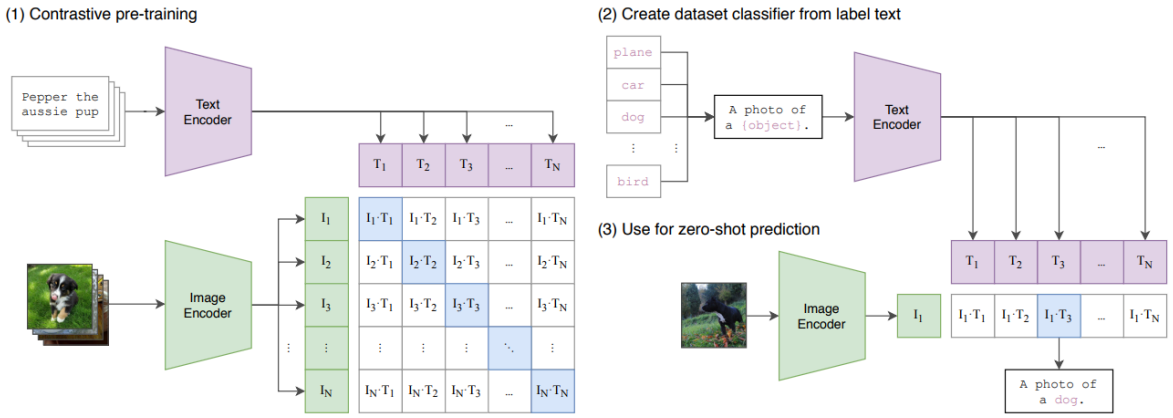


Fig. 1: Overview of CLIP

While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes. [CLIP Paper]

2.2 Domain Generalization (domain shift)

Domain Generalization (DG) aims to improve model generalization on unseen target domains by reducing feature discrepancies across multiple source domains. Common DG techniques include enforcing distribution similarity (e.g., maximum mean discrepancy, Deep CORAL), data augmentation, and domain adversarial learning.

However, existing DG methods in **Facial Expression Recognition (FER)** typically rely on labeled or unlabeled target samples for fine-tuning. In contrast, we propose a **FER model trained on a single dataset**, without access to any target domain samples. This distinguishes it from domain adaptation approaches and makes it more suitable for real-world deployment, where unseen test samples must be recognized without prior domain knowledge.

3. Data

In my **Generalizable facial expression recognition**, I trained the RESNET-18 model on the RAF-DB dataset and tested it on both RAF-DB and FERPlus. Additionally, I trained the model on FERPlus and evaluated its performance on both FERPlus and RAF-DB.

3.1 RAF-DB Dataset

RAF-DB Dataset : (Real-world Affective Faces Database)

RAF-DB is a large-scale facial expression dataset designed for real-world affective computing and facial expression recognition (FER). It contains a diverse set of images with different facial expressions captured under real-world conditions.

Key Features :

- **Size**: 29,672 facial images
- **Labels**: 7 basic emotions (Anger, Disgust, Fear, Happiness, Neutral, Sadness, Surprise)
  Source: Images collected from the Internet
- **Annotation**: Labeled by 40 trained human coders through crowdsourcing.
- **Variability**: Includes images with different ages, genders, ethnicities, occlusions, and lighting conditions.

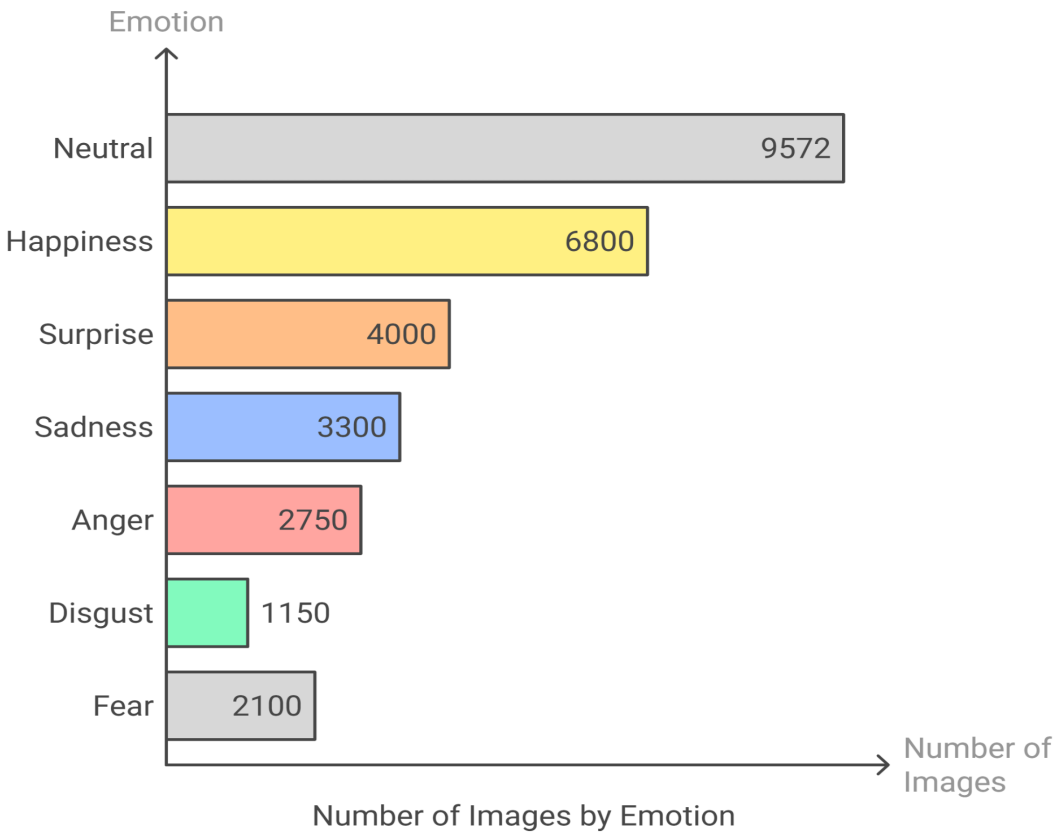


Fig 1 : RAF-DB Dataset Class Diagram

Dataset Structure :

- **Training Set**: 12,271 images
- **Test Set**: 3,068 images

3.2 FERPlus Dataset

FERPlus Dataset : (Facial Expression Recognition Plus)
FERPlus is an improved version of the original **FER2013 dataset**, which was created for the ICML 2013 facial expression recognition challenge. FERPlus enhances the dataset by refining labels through crowd-based annotation.

Key Features :

- **Size**: 35,887 grayscale images (48×48 pixels)
- **Labels**: 8 emotions (Anger, Disgust, Fear, Happiness, Neutral, Sadness, Surprise, Contempt)
- **Source**: Originally from FER2013, labeled using Google's image search
- **Annotation**: Multiple annotators per image, capturing emotion distribution rather than a single label

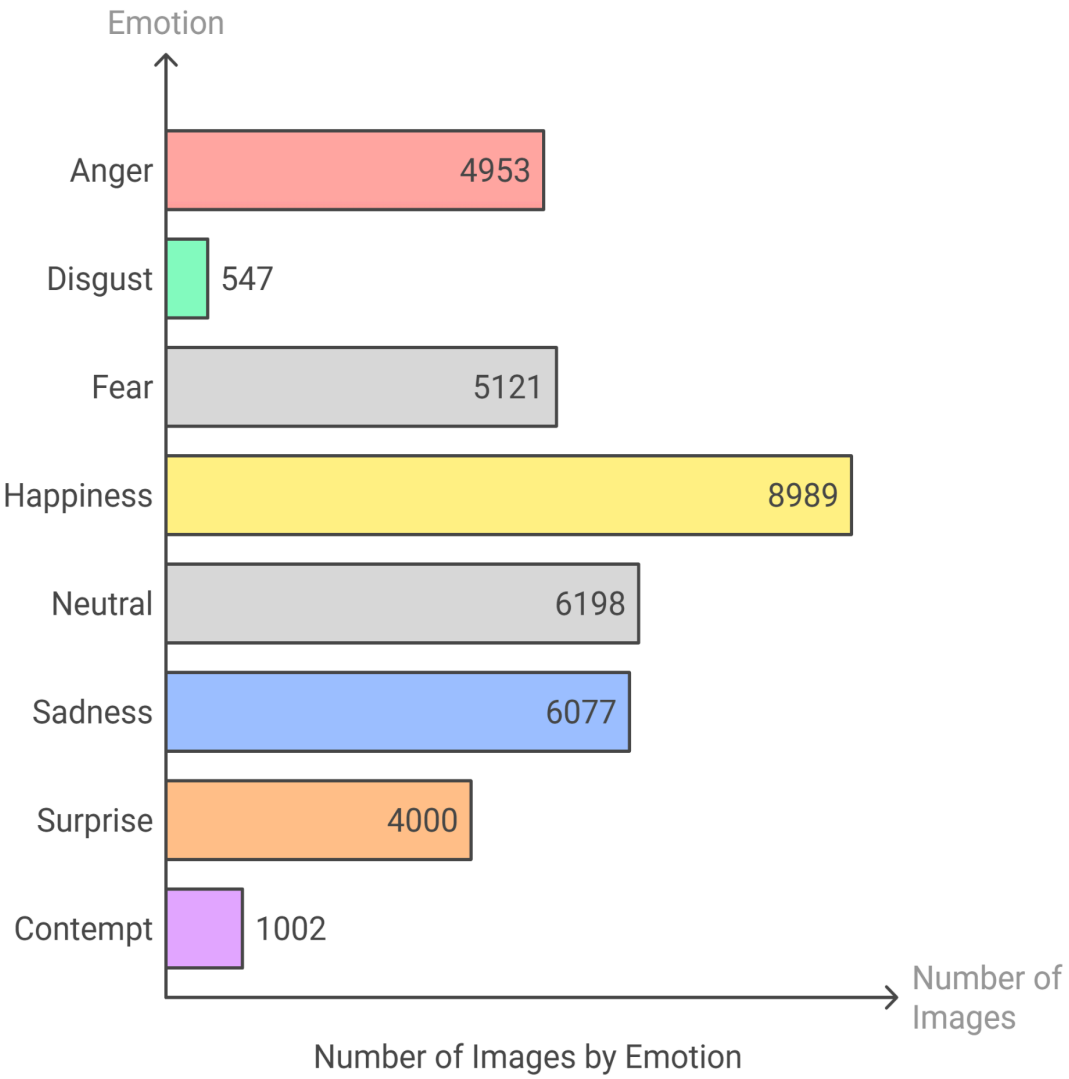


Fig 2 : FERPlus Dataset Class Diagram

Dataset Structure :

- **Training Set**: 28,709 images
- **Validation Set**: 3,589 images
- **Test Set**: 3,589 images

3.3 Data Processing

3.3.1 Adding noise :

```
def add_g(image_array, mean=0.0, var=30):
    std = var ** 0.5
    image_add = image_array + np.random.normal(mean, std, image_array.shape)
    image_add = np.clip(image_add, 0, 255).astype(np.uint8)
    return image_add
```

The add_g function is designed to add **Gaussian noise** to an image for the purpose of **data augmentation**. It works by generating random noise using a Gaussian (normal) distribution with a given mean and variance.
It makes the model **less sensitive to small changes** in input, improving performance on unseen data.
It helps prevent **overfitting** by introducing slight randomness to training samples.
Each pixel's value in the image is changed slightly by adding a **small random number** from a normal distribution.

3.3.2 : Flip the image

```
def flip_image(image_array):
    return cv2.flip(image_array, 1)
```

The flip_image function, on the other hand, performs a **horizontal flip** of the input image using OpenCV's cv2.flip function. This means the image is mirrored left to right. This technique is also a form of data augmentation that helps the model learn **invariance to orientation**, especially useful in tasks like facial expression recognition where left-right symmetry is common

4. Methodology

Overview of CAFE

To address the challenges in facial expression recognition (FER) across different domains, we propose **CAFE** (Cognition of humAn for Facial Expression). Inspired by human cognition, where we **Detect Faces First**, Then **Extract Expression Features**. CAFE extracts facial features and applies a trained FER model to selectively mask expression-related features, ensuring robust emotion recognition.

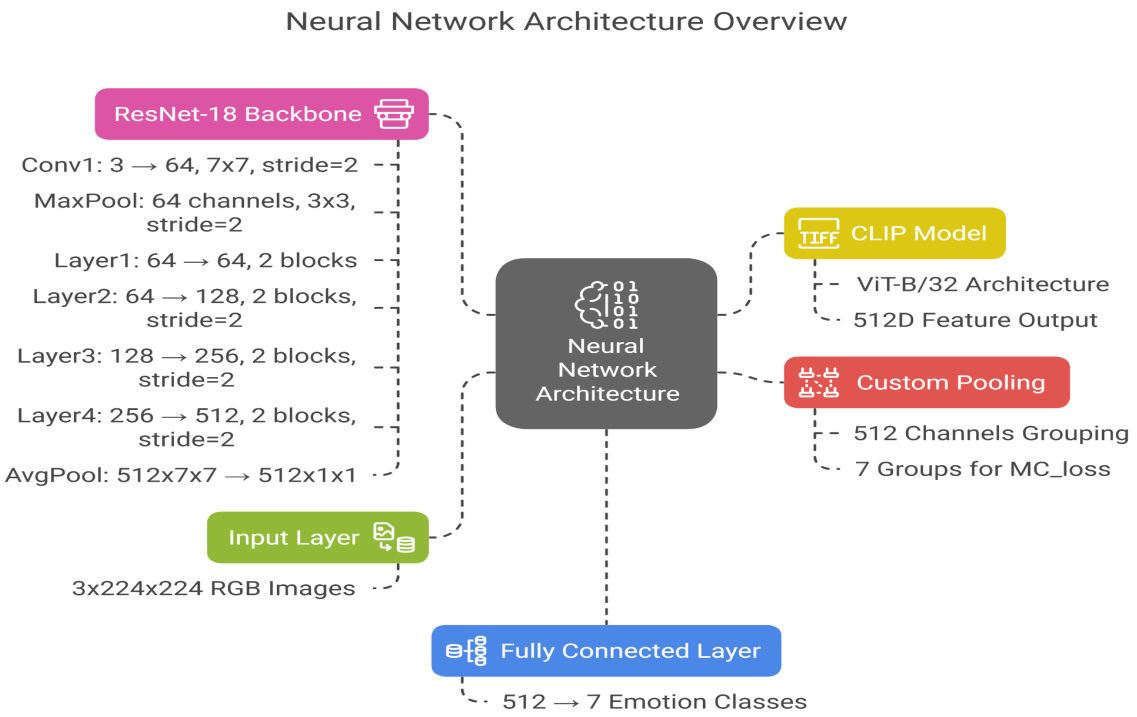


Fig 3 : Flow of my FER Pipeline

4.1 Fixed Face Feature Extraction

```
import clip
device = torch.device('cuda:0')
clip_model, preprocess = clip.load("ViT-B/32", device=device)
```

We are loading the CLIP Model using the CLIP Library.

We utilize a **pre-trained large model**, such as **CLIP**, to extract fixed face features from input images. ViT-B/32 , A **Vision Transformer (ViT) model** with a **Base (B) architecture** that processes images by splitting them into **32×32 pixel patches**, balancing efficiency and performance in tasks like image recognition and multimodal learning.

```
image_features = clip_model.encode_image(x)
```

We are **extracting image features** using CLIP. It converts the input images into a high-dimensional feature vector that represents the image's semantic content.

4.2 Masking Expression-Related Features

Fer Model learns a mask that selects only the expression - relevant features from the general face features. The sigmoid activation function is applied to the mask to regularize its values between 0 and 1. The masked feature representation is used for classification.

Not all facial features contribute to expression recognition. The mask selects only important expression related features,preventing overfitting to domain - specific characteristics.

4.3 channel separation for expression classification

Masked features are separated into 7 groups, each corresponding to one of seven basic expressions. Instead of using a fully connected layer , the model directly transforms the masked features into class logits.

$$\mathcal{L}_{div} = \frac{1}{N(N-1)} \sum_{i=1}^{N} \sum_{j=1, j \neq i}^{N} \frac{h_i^e \cdot h_j^e}{\|h_i^e\| \cdot \|h_j^e\|}$$

Where

N : Number of pooling branches

hi^e : expression-specific feature from branch i.

4.4 Channel diverse Loss

Channel diverse loss is introduced to ensure that each specific feature is distinct from others. This loss forces the model to distribute features across different channels, rather than depending too much on a small subset of features. Without this loss, the model may over rely on certain feature channels, reducing generalization.

$$\mathcal{L}_{sep} = \frac{1}{d} \sum_{i=1}^{d} \left| \rho \left(h_i^e, h_i^i \right) \right|$$

Where

hi^e and hi^i : the i-th channel of expression and identity features.

P : pearson correlation coefficient

D : number of channels

5. Implementation Details :

5.1 Model Architecture : RESNET - 18 (resnet-18)

ResNet-18 is a **Convolutional Neural Network** with **18 layers** deep and is known for introducing **residual connections (skip connections)** to tackle the **vanishing gradient** problem in deep networks.

```
Input Image
     ↓
7x7 Convolution
     ↓
Max Pooling
     ↓
Residual Block 1 (Conv2_x)
     ↓
Residual Block 2 (Conv3_x)
     ↓
Residual Block 3 (Conv4_x)
     ↓
Residual Block 4 (Conv5_x)
     ↓
Global Average Pooling
     ↓
Fully Connected Layer
     ↓
Output Classes
```

```python
def forward(self, x):

    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)

    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)

    x = self.avgpool(x)
    h = x.view(x.shape[0], -1)
    x = self.fc(h)

    return x, h
```

Step : 2

The image is passed through Basic Block.

Input → [3×3 Conv → BN → ReLU]

Extracts low-level features like edges and textures, normalizes them, and adds non-linearity.

[3×3 Conv → BN]

Refines the features further while keeping them normalized, but without non-linearity yet.

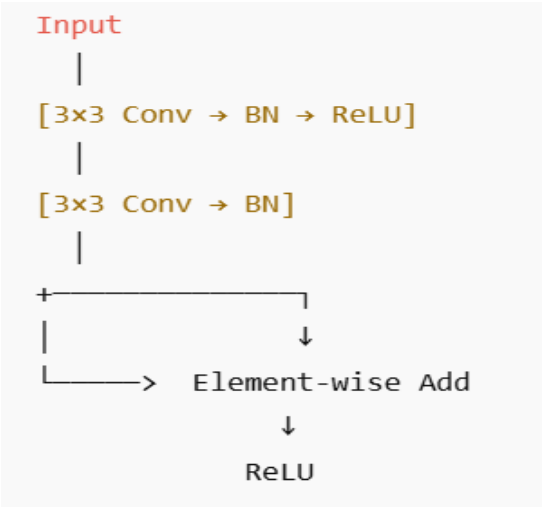
→ Element-wise Add (with Input)

Combines the refined features with the original input to preserve information and help gradients flow (residual connection).

→ ReLU

Applies non-linearity to the combined features to enable learning of complex patterns.

We get one vector x from this.



Step : 3

We send the image x vector through four residual layers of resnet model.

Layer 1 operates on a 56×56 feature map with 64 channels. It contains two residual blocks that preserve spatial resolution. These blocks focus on learning **basic low-level features** like edges, textures, and simple shapes. Since there's no downsampling here, fine details are preserved, which is crucial for initial visual understanding.

Layer 2 reduces the resolution to 28×28 and increases channels to 128. The first block here down samples the input using a convolution with stride 2. This stage starts capturing **mid-level features** such as facial parts — eyes, nose, and mouth — while also reducing computational load. The residual connection includes downsampling to match dimensions.

Layer 3 continues the process, reducing the spatial size to 14×14 and increasing channels to 256. Like before, the first block down samples the input. This stage focuses on learning **higher-level interactions between features**, such as the configuration and spatial relationships between facial components, which are important for recognizing different

expressions.

Layer 4 is the final residual stage, outputting a 7×7 feature map with 512 channels. It captures **the most abstract and semantic features**, combining all prior information into a compact, deep representation of the entire face. These features are now rich enough to distinguish between different facial expressions like happiness, sadness, anger, etc.

Step : 4
The image vector is passed through global average pooling

- Instead of flattening the full feature map (which would have too many parameters), **global average pooling (GAP)** reduces **each channel to 1 number**.
- GAP keeps the model **light**, **interpretable**, and reduces **overfitting**.
- It retains **semantic information** from each feature map (e.g., "smiling", "angry", etc. in facial recognition tasks).

Step : 5
The image vector is passed through a fully connected layer where we get the vector of score (logits)for each class.

5.2 Features Extracted by CLIP

Step 1 :
The processed image passed through CLIP which uses Vision Transformer (ViT-B/32) which extracts image features. CLIP outputs a **512-dimensional image feature vector**, representing the image in a semantic space. VIT-B/31 because the image is passed in batch size -32. These are stored in a vector x.

```
image_features = clip_model.encode_image(x)
```

5.3 Resnet Layers

Step 2:
The Resnet model gives output as 1000 classes now i should specify no of classes and output as number of classes.

```
        self.features = nn.Sequential(*list(res18.children())[:-2])
        self.features2 = nn.Sequential(*list(res18.children())[-2:-1])

        fc_in_dim = list(res18.children())[-1].in_features
```

```
# original fc layer's in dimension 512
        self.fc = nn.Linear(fc_in_dim, num_classes)  # new fc layer 512x7
```

The image vector x is passed through the last two layers, leaving average pooling and fully connected layers.

```
        x = self.features(x)
        feat = x

        x = self.features2(x)
        x = x.view(x.size(0), -1)
```

5.4 Sigmoid activation (sigmoid)

```
        if phase=='train':
              MC_loss = supervisor(image_features * torch.sigmoid(x), targets,
cnum=73)
```

The **Sigmoid function** is a type of activation function used in neural networks. It's a mathematical function that maps any real-valued number to a value between **0 and 1**.

Sigmoid activation is applied on image vector x that is output of resnet layers.
Step 3:

```
        x = image_features * torch.sigmoid(x)
        out = self.fc(x)
```

In the model, image_features come from CLIP and represent how CLIP understands the image. The variable x contains features extracted by the ResNet-18 network from the same image.

We apply the **sigmoid activation** to x, which converts the ResNet features into values between 0 and 1. These values act like soft attention scores, indicating how important each part of the ResNet feature is.

Then, we multiply image_features with torch.sigmoid(x). This operation allows the model to **keep or reduce** certain parts of the CLIP features, based on what ResNet finds important.

The line out = self.fc(x) passes the combined features through a fully connected layer to produce a 7-class prediction. It maps high-level features to facial expression class scores.

Losses Calculated for backpropagation and updation of weights:

Classification Loss (Cls. loss):
Compares the predicted class with the true label
Usually done using cross entropy loss
Separation loss:
Calculated by comparing features across different classes in the batch.
Encourages features from **different classes to be more distinct**.
Helps reduce overlap in feature space.
Diverse Loss:
Encourages different neurons (dimensions) to capture **different aspects** of the data.
Promotes **feature diversity**, so the model doesn't overly rely on just a few dimensions.

At last all losses are added and used to perform backpropagation and update the trainable parameters.

5.5 Main Class

```
    model = Model()

    device = torch.device('cuda:{}'.format(args.gpu))
    model.to(device)
#     model = torch.nn.DataParallel(model, device_ids=[0,1,2])

    optimizer = torch.optim.Adam(model.parameters() , lr=0.0002,
weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)
```

```
    best_acc = 0
    for i in range(1, args.epochs + 1):
        train_acc, train_loss = train(args, model, train_loader, optimizer,
scheduler, device)
        test_acc, test_loss = test(model, test_loader, device)
        print('epoch: ', i, 'acc: ', test_acc)
        if test_acc>best_acc:
            best_acc = test_acc
            torch.save({'model_state_dict': model.state_dict(),},
"ours_best.pth")
        torch.save({'model_state_dict': model.state_dict(),},
"ours_final.pth")
        with open('results.txt', 'a') as f:
            f.write(str(i)+'_'+str(test_acc)+'\n')
```

For each training loop the procedure follows:

Training configuration and hyperparameters :

Optimizer : Adam

Adaptive learning rate : adjusts updates for each parameter dynamically

Weight Decay = 0.0001

A regularization technique (also called L2 regularization) that helps prevent overfitting by regularizing weights.

Prevents overfitting by discouraging the model from learning overly complex weights.

Learning rate and scheduler :

Initial Learning Rate (η) = 0.0002

Small learning rate ensures stable convergence and avoid drastic updates

Scheduler: ExponentialLR

Gradually reduces the learning rate over epochs

Gamma = 0.9

At each step the learning rate multiplied by 0.9, gradually reducing it.

Slows down learning over time, which helps the model fine-tune better in later stages of training and avoid overshooting minima.

Loss functions and weights :

Channel-wise Loss Weight = 1.5

Ensures that different feature channels contribute meaningfully to expression recognition.

Diversity Loss Weight = 5

Encourages feature diversity ensuring the model does not focus too much on specific channels and overfit.

6. Results

Training Dataset	Testing Dataset	Accuracy	Accuracy (ours)
RAF-DB	RAF-DB	88.72	84.29
FERPlus	FERPlus	89.51	86.83
RAF-DB	FERPlus	73.16	86.20
FERPlus	RAF-DB	72.91	85.04

7. Observations and Takeaways

This mini-project was extremely valuable in allowing me to delve headfirst into the world of Computer Vision.

7.1 Observations and Learnings

Here are some of the observations I made and what I learned from them:

- Author in my research paper has tested on 5 dataset classes, i tested only on 2 due to availability of dataset classes.
- I trained my model on raf-db dataset and tested on raf-db and ferplus dataset, and trained model on ferplus dataset and tested on ferplus and raf-db dataset.
- My model is overfitting and still there is a chance for improvement and reduce overfitting.

7.2 Further Learning

During my preparation for this project, I had the opportunity to expand my knowledge significantly.

7.2.1 Neural Networks and Deep Learning

I completed the **Neural Networks and Deep Learning** course by Dr. Andrew Ng on Coursera, which provided a solid foundation in deep learning concepts. The course covered:

- An introduction to deep learning and its applications.
- Logistic regression as a neural network for binary classification, along with the implementation of vectorization in Python.
- Shallow neural networks, their structure, different activation functions, and the backpropagation algorithm.
- Deep neural networks, focusing on how they are built and the role of parameters and hyperparameters in their performance.

7.2.2 Convolutional Neural Networks

I also undertook Dr. Andrew Ng's **Convolutional Neural Networks** course on Coursera, which introduced me to the field of computer vision and equipped me with the foundational knowledge necessary for my project. This course covered:

- The fundamentals of convolutional neural networks, including edge detection, filters, strides, convolution operations over volumes, and pooling layers.
- Various CNN architectures such as ResNets, Inception Networks, MobileNets, and EfficientNets, along with techniques like transfer learning and data augmentation.
- Object detection methods using CNNs, including landmark detection, bounding box predictions, the YOLO algorithm, and U-Net architecture.
- Practical applications of CNNs, particularly in face recognition and neural style transfer.

8. Conclusion and Future Scope

We present a new method to enhance zero-shot generalization in Facial Expression Recognition (FER), aiming to perform well on unseen datasets using only one training set. Inspired by the human approach of first detecting faces and then recognizing expressions, the method uses **sigmoid masks** applied to general face features to isolate expression-relevant information. It combines the broad generalization capability of large models with the precision of FER-specific models. Additionally, we introduce **channel-separation** and **channel-diverse** modules to further regularize these masks and improve generalization.
Future work that can be done using alternative ways :

- Generalizable FER leverages pre-trained models like **CLIP** to extract generalizable face features. However, experimenting with different pre-trained models, such as **ViT (Vision Transformer), Swin Transformer, EfficientNet, or ResNet**, may improve feature extraction quality.
- A **multi-label classification** approach could enable the model to recognize mixed emotions instead of forcing a single-label prediction.
- Current loss functions (e.g., **cross-entropy loss**) might lead to overfitting on the training dataset and fail to generalize well to unseen domains.
- **Contrastive loss** or **metric-learning-based loss functions** could improve feature separation and better model generalization.
- **Domain-invariant loss functions** could be explored to further reduce the impact of domain gaps in unseen test sets.

9. References

1. OpenAI. 2021. “CLIP: Connecting text and images.” OpenAI. https://openai.com/research/clip.
2. https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/02196.pdf
3. https://dl.acm.org/doi/10.1145/2993148.2993165 Learning emotion features with generalization capability by domain part
4. https://www.researchgate.net/figure/Original-ResNet-18-Architecture_fig1_336642248 resent-18 model architecture
5. https://www.v7labs.com/blog/neural-networks-activation-functions sigmoid activation
6. https://www.researchgate.net/publication/375843188_Multi-view_domain-adaptive_representation_learning_for_EEG-based_emotion_recognition
7. https://arxiv.org/abs/2402.14095 (zero shot generalization across architectures)
8. https://arxiv.org/pdf/2103.03097 (generalizing to unseen domains)